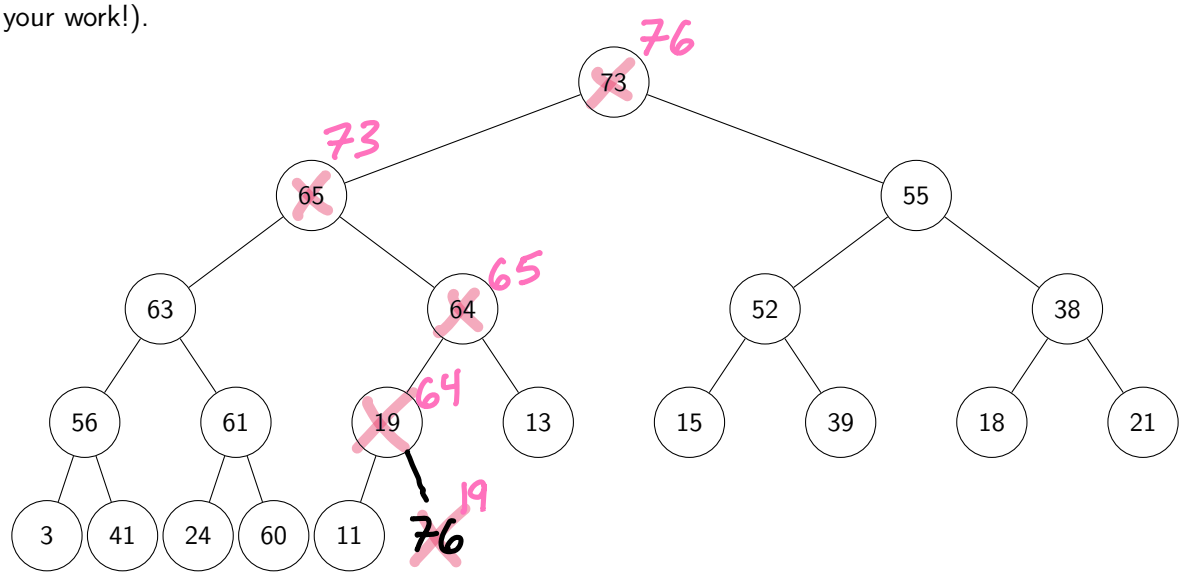
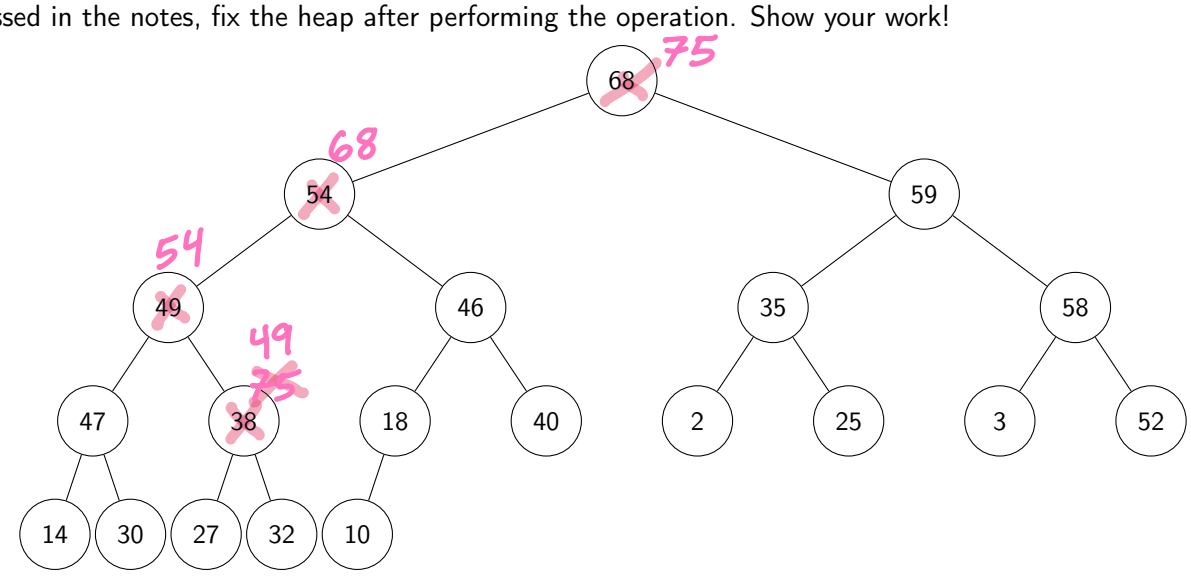
1 Heap Operations

Problem 1.1. Provide a counter example to prove that the following statement is false:
Given a set of values $X = \{x_1, x_2, \ldots, x_n\}$, there is exactly one way to draw a valid max-heap using these values.

Problem 1.2. Apply the `Insert(76)` operation to the following max heap and draw the resulting binary tree (show your work!).



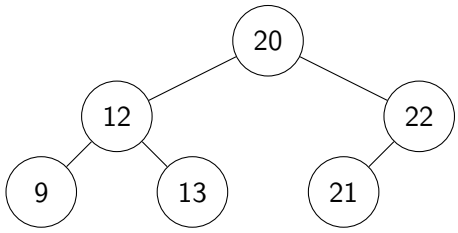
Problem 1.3. Suppose that the value 38 is replaced with the value 75, following the "increase key" procedure discussed in the notes, fix the heap after performing the operation. Show your work!



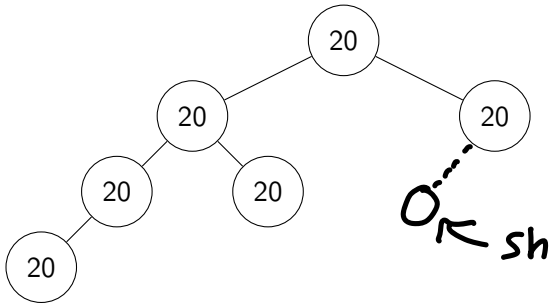
Problem 1.4. Which of the following are valid MAX-heaps? If the binary tree is not a valid max heap, then explain why (briefly! At most two sentences).

(a)

YES

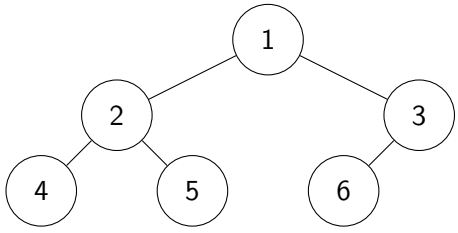
(b) 

NO $22 \geq 20$

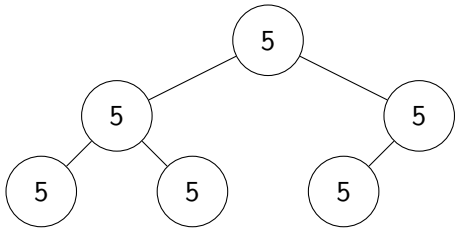
(c) 

NO

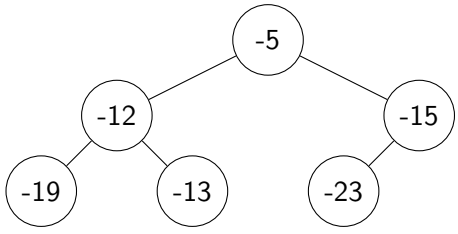
0 ← should be here

(d) 

NO $3 \geq 1$.

(e) 

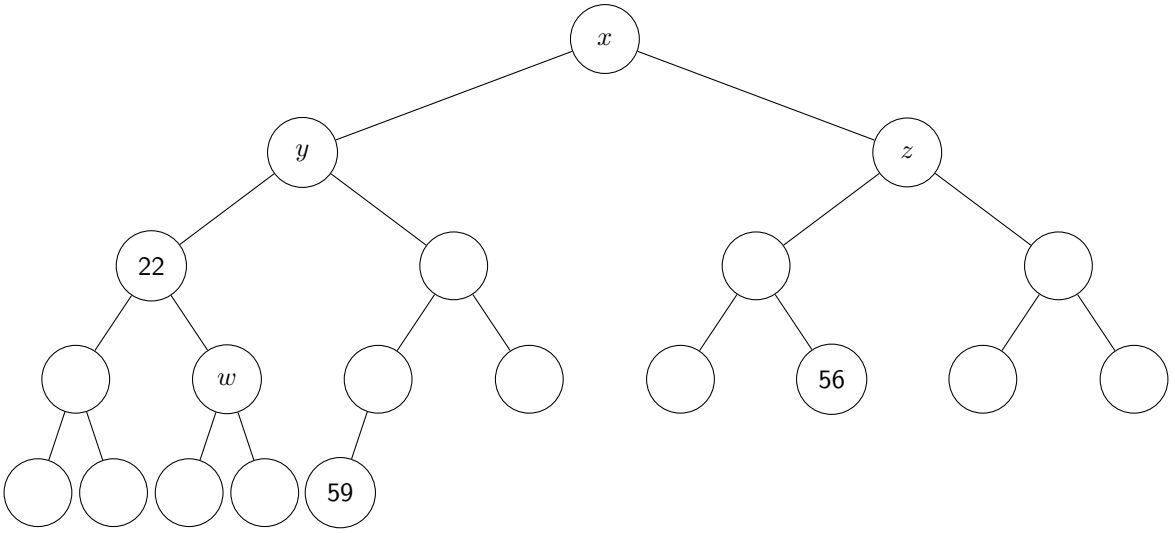
YES

(f) 

YES

(g) YES

Problem 1.5. Consider the following max-heap,



Using only the knowledge of the values present in the tree and the definition of a max-heap, what can we say about the values of x,y,z, and w?

$x \geq 59$ $\quad$ $y \geq 59$ $\quad$ $z \geq 56$

$x \geq y$ $\quad$ $w \leq 22$

2 Analyzing algorithms that use data structures

Problem 2.1. Determine the asymptotic complexity of the following function

```
function Func(A[],n)
    Q.Init()                                   s = 1
    for i = 1 to n/2 do
        for j = 1 to 2n^2 do
            Q.Insert(A[i] · A[j mod n])
        end for
    end for                                    s = n^3
    for i = 1 to n^2 do
        x = Q.ExtractMax()
        Print x
    end for                                    s = n^3 - n^2
end function
```

Assume that Q is a max-heap implemented the same as in the notes.

Analyze this problem by finding the size of the data structure before each loop executes and the size of the data structure after each loop terminates.

$$T(n) = \sum_{s=1}^{n^3} T_{in}(s) + \sum_{s=n^3-n^2}^{n^3} T_{em}(s)$$

$$T(n) = \sum_{s=1}^{n^3} c\log(s) + \sum_{s=n^3-n^2}^{n^3} c\log(s)$$

UB: $T(n) \le \sum_{s=1}^{n^3} c\log(n^3) + \sum_{s=n^3-n^2}^{n^3} c\log(n^3)$

$T(n) \le n^3 \cdot c\log(n^3) + (n^3 - (n^3-n^2) + 1)\, c\log(n^3)$ $\Big\}\ O(n^3\log(n))$

LB: $T(n) \ge \sum_{s=1}^{n^3} c\log(s)$ [drop 2nd sum]

$T(n) \ge \sum_{s=\frac{n^3}{2}}^{n^3} c\log(n^3/2)$

$T(n) \ge (n^3/2 + 1)\, c\log\left(\frac{n^3}{2}\right)$ $\Big\}\ \Omega(n^3\log(n))$

Thus, $T(n) \in \Theta(n^3\log(n))$.

Problem 2.2. Determine the best, worst, and expected case running time of the following function

```
function Func(A[],n)
    H.Init()                                        s = 1
    for i = 1 to n/2 do
        for j = 1 to 2⌊√n⌋ do
            if not H.Member(A[i] · A[j]) then
                H.Insert(A[i] · A[j], true)         } cs
            end if
        end for
    end for                                         s = n^1.5
    for i = 1 to n do
        if H.Member(A[i]) then
            Print "Found A[i]"                      } cs = cn^1.5
        end if
    end for                                         s = n^1.5
end function
```

Assume that H is a hash table implemented using **open address hashing** with size $m > 2n^{3/2}$.

Analyze this problem by finding the size of the data structure before each loop executes and the size of the data structure after each loop terminates.

BC: All methods take $\Theta(1)$ time. $B(n) \in \Theta(n^{1.5})$

EC: All methods take $\Theta(1)$ time $ET(n) \in \Theta(n^{1.5})$

WC: All methods take $\Theta(s)$ time AND we always insert.

$$W(n) = \sum_{s=1}^{n^{1.5}} cs + \sum_{i=1}^{n} cn^{1.5}$$

$$W(n) = c\,\frac{n^{1.5}(n^{1.5}+1)}{2} + n \cdot cn^{1.5}$$

Thus, $W(n) \in \Theta(n^3)$.

Problem 2.3. Determine the asymptotic complexity of the following function

```
function Func(A[],n)
    Q.Init()
    for i = 1 to n/2 do
        for j = 1 to 2i do
            Q.Insert(A[i] · A[j])
        end for
    end for
    for i = 1 to n do
        x = Q.ExtractMax()
        Print x
    end for
end function
```

$s=1$

$s=M$

$s=M-n$

$$\#\text{ of inserts} = \sum_{i=1}^{\frac{n}{2}} \sum_{j=1}^{2i} 1 = \sum_{i=1}^{n/2} 2i = 2 \cdot \frac{\frac{n}{2}\left(\frac{n}{2}+1\right)}{2} = \frac{n}{2} \cdot \left(\frac{n}{2}+1\right)$$

$$\#\text{ of extracts} = n$$

FOR CONVENIENCE, LET $M = \frac{n}{2}\left(\frac{n}{2}+1\right)$

Assume that Q is a priority queue where

- `Q.Init()` initializes the data structure and takes $\Theta(1)$ time.
- `Q.Insert(x)` inserts x into the priority queue and takes $\Theta(\sqrt{s})$ time.
- `Q.ExtractMax()` removes and returns the maximum element in the priority queue and takes $\Theta(s)$ time.

where s is the number of elements in Q.

Analyze this problem by finding the size of the data structure before each loop executes and the size of the data structure after each loop terminates.

$$T(n) = \sum_{s=1}^{M} c\sqrt{s} + \sum_{s=M-n}^{M} cs$$

UB:

$$T(n) \le \sum_{s=1}^{M} c\sqrt{M} + \sum_{s=M-n}^{M} cM = M \cdot c\sqrt{M} + (M-(M-n)+1)cM$$

$$= cM^{1.5} + (n+1)cM$$

$$= \underbrace{c\left[\frac{n}{2}\left(\frac{n}{2}+1\right)\right]^{1.5}}_{\approx n^3} + \underbrace{(n+1)c \cdot \frac{n}{2}\left(\frac{n}{2}+1\right)}_{\approx n^3} \Bigg\} O(n^3).$$

LB: (Drop 1st sum)

$$T(n) \ge \sum_{s=M-n}^{M} cs$$

$$\ge \sum_{s=M-n}^{M} c(M-n) = (n+1)c(M-n) = (n+1)c\left(\frac{n}{2}\left(\frac{n}{2}+1\right) - n\right) \Bigg\} \Omega(n^3).$$

Thus, $T(n) \in \Theta(n^3)$.